

파머완 08 텍스트 분석 part 2

06 토픽 모델링 - 20 뉴스그룹

토픽 모델링: 문서 집합에 숨어 있는 주제를 찾는 것

- 사람이 수행 → 더 함축적인 의미로 문장 요약
- ML 수행 → 숨겨진 주제를 효과적으로 표현하는 중심 단어 추출
- LDA(Latent Dirichlet Allocation)

07 문서 군집화 소개와 실습 (Opinion Review 데이터 세트)

문서 군집화 개념

문서 군집화: 비슷한 텍스트 구성의 문서를 군집화하는 것

- 동일한 군집에 속하는 문서를 같은 카테고리 소속으로 분류 (텍스트 분류 기반 문서 분류와 유사)
- 학습 데이터 세트가 필요없는 비지도학습

Opinion Review 데이터 세트를 이용한 문서 군집화 수행하기

데이터 세트: 51개의 텍스트 파일, 약 100개의 문장

여러 파일을 하나의 dataframe으로 로딩하여 데이터 처리

	filename	opinion_text
0	filename_list	short battery life I moved up from an 8gb .\n0 I love this ipod except for the battery life .\n1 ...
1	filename_list	The voice prompts and maps are wonderful especially when driving after dark .\n0 I also thought the the voice prompts of the 750 where more pleasant sounding than the 255w's .\n1 ...
2	filename_list	3 quot widescreen display was a bonus .\n0 This made for smoother graphics on the 255w of the vehicle moving along displayed roads, where the 750's display was more of a jerky movement .\n1 ...
3	filename_list	, and is very, very accurate .\n0 but for the most part, we find that the Garmin software provides accurate directions, wherever we intend to go .\n1 This functi...
4	filename_list	, I think the new keyboard rivals the great hp mini keyboards .\n0 Since the battery life difference is minimum, the only reason to upgrade would be to get the better keyboard .\n1 The keyboard is now as good as t...

TF-IDF 피쳐 벡터화

TfidfVectorizer()

- 어근 변환 지원 X
- tokenizer 인자에 커스텀 어근 변환 함수 적용 → 어근 변환 수행 가능

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vect=TfidfVectorizer(tokenizer=LemNormalize,stop_words='english',ngram_range=(1,2))
# opinion_text 칼럼값으로 feature vectorization 수행
feature_vect=tfidf_vect.fit_transform(document_df['opinion_text'])
```

5개의 중심 기반으로 군집화 확인

```
from sklearn.cluster import KMeans

# 5개 집합으로 군집화 수행. 예제를 위해 동일한 클러스터링 결과 도출용 random_state
km_cluster=KMeans(n_clusters=5,max_iter=10000,random_state=0)
km_cluster.fit(feature_vect)
cluster_label=km_cluster.labels_
cluster_centers=km_cluster.cluster_centers_
```

cluster #0 - 호텔에 대한 리뷰

cluster #1 - 포터블 전자기기 및 주요 구성요소에 대한 리뷰

cluster #2 - cluster #1 + 차량용 내비게이션 리뷰

cluster #3 - 킨들 리뷰 1개 + cluster #0

cluster #4 - 자동차에 대한 리뷰

⇒ 군집 개수가 많게 설정되어 세분화된 군집화 발생

3개의 중심 기반으로 군집화 재확인

```
from sklearn.cluster import KMeans

# 3개의 집합으로 군집화
km_cluster = KMeans(n_clusters=3, max_iter=10000, random_state=0)
km_cluster.fit(feature_vect)
cluster_label = km_cluster.labels_

# 소속 클러스터를 cluster_label 컬럼으로 할당하고 cluster_label 값으로 정렬
document_df['cluster_label'] = cluster_label
document_df.sort_values(by='cluster_label')
```

cluster #0 - 포터블 전자기기 리뷰

cluster #1 - 자동차 리뷰

cluster #2 - 호텔 리뷰

⇒ 군집이 잘 구성됨

군집별 핵심 단어 추출하기

clusters_centers: 각 군집을 구성하는 단어 피처가 centroid를 기준으로 얼마나 가깝게 위치해 있는지 제공

```
##### Cluster 0
Top features: ['room', 'hotel', 'service', 'staff', 'interior', 'food', 'location', 'seat',
Reviews 파일명 : ['filename_list', 'filename_list', 'filename_list', 'filename_list', 'f
=====
##### Cluster 1
```

```

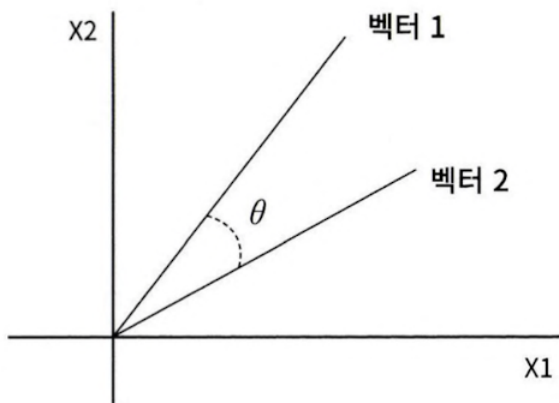
Top features: ['battery', 'battery life', 'keyboard', 'life', 'screen', 'video', 'perform
Reviews 파일명 : ['filename_list', 'filename_list', 'filename_list', 'filename_list', 'f
=====
##### Cluster 2
Top features: ['screen', 'direction', 'voice', 'map', 'kindle', 'button', 'page', 'feat
Reviews 파일명 : ['filename_list', 'filename_list', 'filename_list', 'filename_list', 'f
=====

```

08 문서 유사도

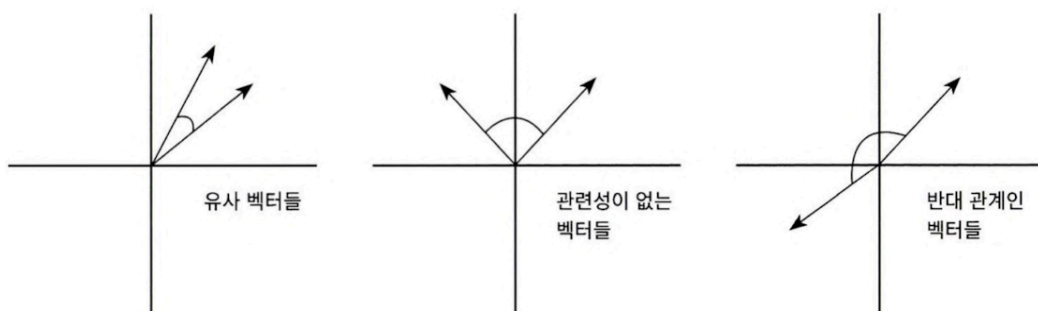
문서 유사도 측정 방법 - 코사인 유사도

코사인 유사도: 벡터와 벡터 간 유사도를 비교할 때 벡터의 상호 방향성이 얼마나 유사한지에 기반함



⇒ 두 벡터 사이의 사잇각을 구해서 얼마나 유사한지 수치로 적용함

두 벡터 사잇값



두 벡터 내적 값

$$A \cdot B = \|A\| \|B\| \cos \theta$$

유사도

$$\text{similarity} = \cos \theta = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

cos_similarity() 함수 정의

```
import numpy as np

def cos_similarity(v1, v2):
    dot_product = np.dot(v1, v2)
    l2_norm = (np.sqrt(sum(np.square(v1))) * np.sqrt(sum(np.square(v2))))
    similarity = dot_product / l2_norm

    return similarity
```

두 문서의 유사도 측정

```
# TFidfVectorizer로 transform()한 결과는 Sparse Matrix이므로 Dense Matrix로 변환
feature_vect_dense = feature_vect_simple.todense()

#첫번째 문장과 두번째 문장의 feature vector 추출
vect1 = np.array(feature_vect_dense[0]).reshape(-1,)
vect2 = np.array(feature_vect_dense[1]).reshape(-1,)

#첫번째 문장과 두번째 문장의 feature vector로 두개 문장의 Cosine 유사도 추출
similarity_simple = cos_similarity(vect1, vect2)
print('문장 1, 문장 2 Cosine 유사도: {0:.3f}'.format(similarity_simple))
```

문장 1, 문장 2 Cosine 유사도: 0.402

cosine_similarity API

```
from sklearn.metrics.pairwiseimport cosine_similarity

similarity_simple_pair= cosine_similarity(feature_vect_simple[0] , feature_vect_simple)
print(similarity_simple_pair)
```

두 문서 유사도 측정

```
from sklearn.metrics.pairwise import cosine_similarity

similarity_simple_pair = cosine_similarity(feature_vect_simple[0] , feature_vect_simple)
print(similarity_simple_pair)
```

[[0.40207758 0.40425045]]

09 한글 텍스트 처리 - 네이버 영화 평점 감성 분석

한글 NLP 처리의 어려움

1. 띄어쓰기 문제

- 띄어쓰기 하나로 의미가 완전히 바뀔 수 있음
(예: "아버지가 방에 들어가신다" vs "아버지 가방에 들어가신다")
- 영어는 띄어쓰기 오류 시 의미 왜곡보다는 단어 인식 오류로 끝남
- 한글은 고등교육자도 종종 띄어쓰기 실수함

2. 조사의 다양성

- 주어나 목적어 뒤에 붙는 조사(은, 는, 이, 가, 에서 등)의 종류가 매우 다양

- 어근 추출이나 불용어 제거 시 처리가 복잡함
- '은' 같은 경우 조사인지 단어 일부인지 판단도 어려움
(예: '집은' → 조사 '은'인지, 금속 '은(銀)'인지)

⇒ 한글은 과학적인 문자이지만, **띄어쓰기와 조사 처리 문제 때문에 영어보다 NLP 전처리가 더 어려움**

KoNLPy

: 파이썬의 대표적인 한글 형태소 패키지

TF-IDF 벡터화 - twitter 클래스 사용

```
from konlpy.tag import Twitter

twitter = Twitter()
def tw_tokenizer(text):
    # 입력 인자로 들어온 text 를 형태소 단어로 토큰화 하여 list 객체 반환
    tokens_ko = twitter.morphs(text)
    return tokens_ko
```

TfidfVectorizer → TF-IDF 피쳐 모델 생성

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import GridSearchCV

# Twitter 객체의 morphs( ) 객체를 이용한 tokenizer를 사용. ngram_range는 (1,2)
tfidf_vect = TfidfVectorizer(tokenizer=tw_tokenizer, ngram_range=(1,2), min_d
tfidf_vect.fit(train_df['document'])
tfidf_matrix_train = tfidf_vect.transform(train_df['document'])
```

로지스틱 회귀 → 분류 기반 감성 분석

```
# Logistic Regression 을 이용하여 감성 분석 Classification 수행.
lg_clf = LogisticRegression(random_state=0, solver='liblinear')
```

```
# Parameter C 최적화를 위해 GridSearchCV 를 이용.
params = { 'C': [1 ,3.5, 4.5, 5.5, 10 ] }
grid_cv = GridSearchCV(lg_clf , param_grid=params , cv=3 ,scoring='accuracy')
grid_cv.fit(tfidf_matrix_train , train_df['label'] )
print(grid_cv.best_params_ , round(grid_cv.best_score_,4))
```

output)

```
Fitting 3 folds for each of 5 candidates, totalling 15 fits
{'C': 3.5} 0.8593
```

테스트 세트 → 최종 감성 분석 예측

```
from sklearn.metrics import accuracy_score

# 학습 데이터를 적용한 TfidfVectorizer를 이용하여 테스트 데이터를 TF-IDF 값으로 Fe
tfidf_matrix_test = tfidf_vect.transform(test_df['document'])

# classifier 는 GridSearchCV에서 최적 파라미터로 학습된 classifier를 그대로 이용
best_estimator = grid_cv.best_estimator_
preds = best_estimator.predict(tfidf_matrix_test)

print('Logistic Regression 정확도: ',accuracy_score(test_df['label'],preds))
```

output)

```
Logistic Regression 정확도: 0.86172
```